Ethics 1: Team Conclusions

Unethical software systems: discussion summary

1. Boeing 737 MAX (MCAS flight control software)

MCAS was software added to the 737 MAX to push the nose down automatically when it thought the plane was near a stall. It read from a single angle-of-attack sensor, so one faulty reading could trigger repeated nose-down commands. Pilots were not told the system existed, and early training materials left it out. Two crashes (Lion Air 610 in 2018 and Ethiopian Airlines 302 in 2019) killed 346 people.

- **Why it's problematic:** The design had no redundancy for a safety-critical input, the system's authority grew during development, and the people flying the plane could not respond to a system they didn't know about. Pressure to compete with Airbus and avoid costly pilot retraining pushed the problem out of sight.
- **Engineer or product:** The function itself (stabilizing handling) was legitimate, so this isn't a cheat device. The failures were a single point of failure in the design, downplaying the system to regulators and customers, and a schedule and cost culture that overrode engineers' concerns. Individual engineers were working inside decisions made above them.
- **Why it seems unethical:** It broke the duty of care owed to people whose lives depend on the system. Passengers and pilots couldn't consent to a risk they weren't told about, and cost and schedule were put ahead of safety.
- **Conclusions:** Safety-critical software needs redundancy, honest disclosure to the people who depend on it, and engineers who can raise concerns without being overruled by schedule. A system hidden from its operators is unsafe even if every line of code does what it was specified to do.

2. Volkswagen emissions software ("Dieselgate")

Engine control software detected test conditions (steering input, speed profile, duration) and turned on full emissions controls only then. On the road, the cars emitted up to roughly 40 times the legal NOx limit.

- **Why it's problematic:** It is deliberate deception of regulators and buyers, with real health costs from air pollution. The code existed for no purpose other than cheating the test.
- **Engineer or product:** Both, and the engineer side is clear. Engineers wrote and maintained code whose only function was to detect and defeat a test, and several were criminally convicted. The product function (a cheat device) was itself unethical, so there is no "neutral tool" defense.
- **Why it seems unethical:** It is built-in dishonesty. The software existed to lie to regulators and customers, and it knowingly shifted a health cost (air pollution) onto the public for profit.
- **Conclusions:** "Management told me to" doesn't hold up when you can see exactly what your code does. This is the standard case for why engineers carry personal responsibility and need a channel to refuse or report.

3. Facebook data harvesting

A quiz app collected data from about 270,000 users, and through the platform's friend-data permissions it also pulled profiles of tens of millions of their friends who never consented. The data was then used for political ad targeting.

- **Why it's problematic:** Consent was effectively absent for most affected people. The data was used for a purpose far from what anyone agreed to, and Facebook's API made this possible by design.

- **Engineer or product:** The permissive API was a product and policy decision. The researcher who built the app broke the stated terms. Individual engineers likely did nothing individually wrong, yet the system as a whole did harm.
- **Why it seems unethical:** It violated informed consent and the expectation that data is used only for the purpose it was shared for. Tens of millions of people were affected without ever agreeing to anything.
- **Conclusions:** This is a systemic failure, not a single bad actor. Harm emerged from the combination of an engagement-driven data model, weak enforcement, and downstream misuse. "We followed the API rules" isn't an ethical defense when the rules themselves enable abuse.